Introduction à la Programmation et l'Algorithmique

DINO LOPEZ PACHECO

DINO.LOPEZ@UNIV-COTEDAZUR.FR

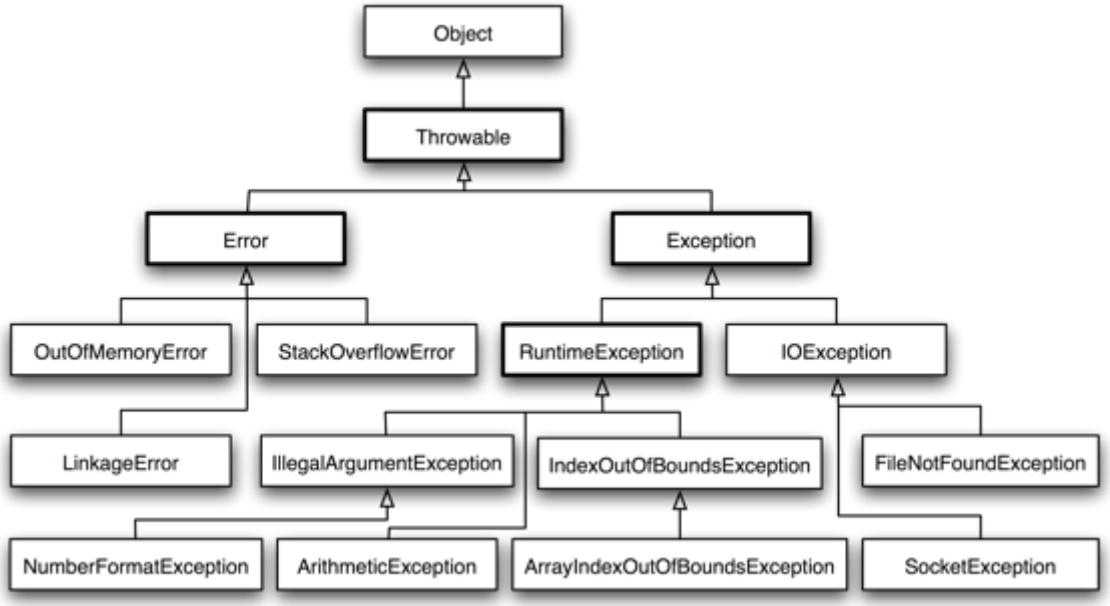
Administration d'Exceptions

Principe générale des Exceptions

- Les langage de programmation à haute niveau emploient souvent des Exceptions pour gérer des erreurs
  - Éviter les arrêts intempestives des programmes.
- Une exception est un mécanisme qui casse la séquentialité d'un programme
  - Rappel : « return », « break » ou « continue » cassent également cette séquentialité
  - Levée d'exception -> redirection vers le gestionnaire d'exceptions
- L'exception se propage depuis la méthode ou la fonction où l'erreur se produit, et remonte la chaîne des méthodes ou fonctions appelants.
  - A chaque fois qu'un gestionnaire d'exception est trouvé, l'exception levée vérifie si cette dernière appartient au type géré par le gestionnaire
  - Le gestionnaire gérant l'exception levé exécutera son code

La classe « Throwable »

- En Java, tous les exception héritent de la classe « Throwable »
- 3 catégories d'exceptions
  - Erreurs critiques (étendent la classe « Error »), pas censé être gérés normalement
  - Erreurs qui doivent être gérés par le programme -> erreur de compilation si ce n'est pas fait (e.g. IOException)
    - « Exception »
    - Non RunTimeException's
  - Erreurs lors de l'exécution du programme
    - RuntimeException
    - Peut-être géré par le programme – si on veut…



Lancement d'Exceptions

- 2 ou 3 entités entrent en jeu
  - Le point où l'erreur se trouve (lanceur de l'exception)
  - Les points intermédiaires remontant l'exception
    - Exception devant être gérés (e.g. IOException)
  - Le point où le gestionnaire de l'exception se trouve
- Lancement d'une exception
  - Mot clé « throw »
  - L'exception étant un objet, il faut créer l'exception avec « new »
  - Seulement les classes héritant directement ou indirectement de « throwable » peuvent être « thow »

- Exemple

```
// on demande une chiffre entre 0 et 9
int val = scanner.nextInt();
if (val > 9) {
    throw new ExceptionGrandInt();
}
```

Management des exceptions

Traiter une exception

- Les exceptions levées sont traités par les blocs « try » et « catch »

```
try {
    // code qui peut déclencher une exceptio
} catch (TypeException e) {
    // code à exécuter si l'exception indiquée
    // en haut corresponde à l'exception levé
    // (si l'exception est rattrapée)
}
```

Faire passer une exception

- Une exception levée mais qui n'est pas traité par la méthode locale doit être renvoyé
  - Mot clé « throws »
- Exemple

```
public int getInt() throws ExceptionGrandInt {
  int val = scanner.nextInt();
  if (val > 9) {
    throw new ExceptionGrandInt();
  }
  return val;
}
```

Exemples additionnels de « throws »

```
public void appelGetInt() throws
ExceptionGrandInt {
   …
   int val = obj.getInt();
   …
   // notez l'absence du try…catch
pour gérer ExceptionGrandInt
}
```

```
public void anyMethod() throws
ExceptionGranInt, ExceptionOne,
ExceptionAutre {
…
}
```

Gestion des exceptions

- Blocs « try, catch »
  - Possibilité d'écrire des multiples de bloques « catch » pour un seul try
  - « try, catch » dans un « try, catch »
- L'exécution du bloc « try » s'arrête sur la ligne de code donnant lieu à l'exception
  - Puis le gestionnaire d'exception prend la main (i.e. le bon « catch »)
- Si des multiples « catch » sont empilés, il faut toujours commencer par l'exception la plus particulier et finir par la plus générale
  - Le premier catch attrapant l'exception « consomme » l'exception

```
try {
  // tab.length == 3
  y = 0;
  int val = tab[3];
  y += 10;
} catch (ArrayIndexOutOfBoundException e) {
…
} catch (IndexOutOfBoundException e) {
…
} catch (RuntimeException e) {
…
}
// Quelle valeur aura y ?
// Que se passe-t-il si le catch avec
RuntimeException apparait en 1er ?
```

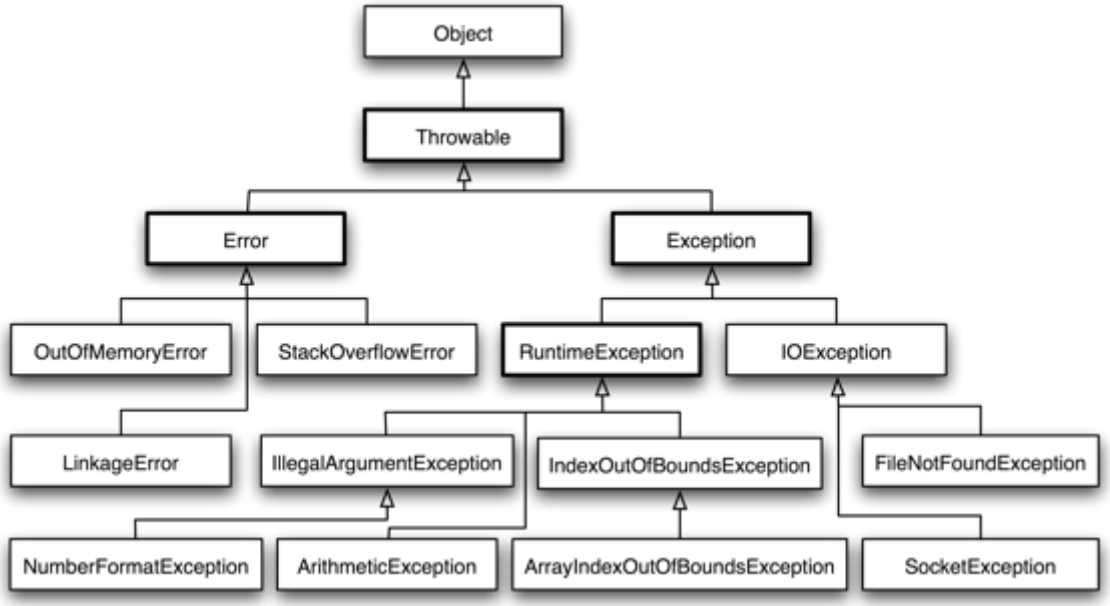
Exception et héritage

- Lors d'une substitution (@Override), la sous-classe
  - Peut ajouter la clause « throws » à une méthode
    - si cette dernière possède la même clause « throws » au niveau de la super-classe
    - Même si cette dernière ne possède pas la même clause « throws » au niveau de la super-classe, à condition que l'Exception soit de type « RuntimeException » ou « Error ».
  - Peut omettre la clause « throws » seulement si l'exception signalée au niveau de la super-classe est de type « RuntimeException » ou « Error »

Création d'exceptions

Écriture d'une exception

- On doit écrire un classe.
  - décider s'il s'agit d'une erreur critique (pas à être géré par le développeur)
    - Étendre la classe « Error »
  - Erreur devant être géré par le développeur → sinon, compilation KO
    - Étendre « Exception » ou une autre sous-classe (sauf RuntimeException), selon le cas
  - Erreur d'exécution -> erreur légère
    - Étendre « RunTimeException » ou une autre sous-classe
- Notre classe d'exception contiendra autant de méthodes que nécessaire pour debugger l'erreur



Exemple avec sous-classe de RuntimeException

Levée de l'exception

```
public class AnyClass {
  …
  public int getInt() throws
ExceptionGrandInt {
    …
    int val = scanner.nextInt();
    if (val > 9) {
      throw new
ExceptionGrandInt(val);
    }
    return val;
  }
…
}
```

Gestion de l'exception

```
public class AnyOtherClass {
  …
  public void utiliseGetInt() {
    …
    try {
    …
      int val = ac.getInt();
    …
    } catch (ExceptionGrandInt e)
{
      System.out.println(e);
    }
  }
  …
}
```

La nouvelle exception

```
public class ExceptionGrandInt
extends RuntimeException {
  private int val;
  public ExceptionGrandInt(int
val) {
    this.val = val;
  }
  public int getVal() {
    return val;
  }
  public String toString() {
    return(val +  " is a Big
Int!!");
  }
}
```

finally

Libération de ressources

- Lorsqu'on bloque une ressource (e.g. un fichier en écriture/lecture), il faut la libérer proprement avant la fin de notre programme (ou méthode)
  - Les ressources non libérées peuvent conduire à de blocages sur notre et autres programmes
  - Des erreurs (soulevant des exceptions), des instructions « continue », « break » ou « return » peuvent conduire à un problème de blocage de ressources
- La clause « finally » est une clause qui est toujours exécutée, même en présence d'évènements cassant la séquentialité du programme
  - Excellent candidat pour libérer les ressources bloquées par un programme
- Le bloc « finally » s'écrit après un bloc « try, catch » un après un bloc « try »

Exemples

```
try {
} catch () {
} finally {
}
```

```
try {
} finally {
}
```

break vs continue

break vs continue

- « break » casse la séquentialité pour sortir d'une boucle
- Exemple

```
int i = 0;
while true {
  if (i >= 100) break;
  i++;
}
// i = ??
```

- « continue » casse la séquentialité du programme pour exécuter une nouvelle entrée dans boucle
- Exemple

```
int i = 0;
while (i < 100) {
  i++;
  if ((i % 2) == 0) continue;
  System.out.println(i);
}
// sortie de println() ??
```